

Wednesday, February 11th, 2026

Long-read RNA-seq Analysis — Part 2.1

Cleanup Bambu output files and generate overview about the assembled transcriptome

Vinicius Maracaja-Coutinho ^{*,1,2,3} and Bárbara Borges ^{†,1,2}

***Principal Investigator** — Bioinformatics PhD

†Biotechnologist — Bioinformatics MSc(c)

¹Laboratory of Integrative Bioinformatics (LIB), Unidad de Genómica Avanzada, Universidad de Chile

²Bioinformatics Multidisciplinary Environment (BioME), UFRN, Brazil

³Laboratory of Precision Medicine and Health, Fiocruz Salvador, Brazil

Generating the transcriptome overview

Let's get an overview of how the assembly went by calculating the total number of genes and transcripts. How many transcripts assembled are already annotated? How many of them reached full-length and uniquely mapped structures?

First, you will run this bash script to remove genes and transcripts that received 0 counts across all samples.

```
# Filter out zero-count rows.
# This script reads the original count/CPM files and removes any
# gene or transcript where all sample columns are 0.

# --- Configuration and File Paths ---

# Define input files (should be in the same directory as the script)
SCRIPT_DIR="$HOME/bambu"

# Input files to process
declare -a INPUT_FILES=(
    "${SCRIPT_DIR}/BambuOutput_counts_gene.txt"
    "${SCRIPT_DIR}/BambuOutput_counts_transcript.txt"
    "${SCRIPT_DIR}/BambuOutput_CPM_transcript.txt"
    "${SCRIPT_DIR}/BambuOutput_fullLengthCounts_transcript.txt"
    "${SCRIPT_DIR}/BambuOutput_uniqueCounts_transcript.txt"
)

# --- Argument Handling ---

if [ "$#" -ne 0 ]; then
    echo "Usage: $0"
    echo "This script filters predefined input files for zero-count rows."
    echo "Files processed:"
    for file in "${INPUT_FILES[@]}; do
        echo "  - $(basename "$file")"
```

```

done
exit 1
fi

# --- Processing Functions ---

# Function to process gene count file (checks cols 2-NF)
process_gene_file() {
    local input_file="$1"
    local output_file="$2"

    echo "Processing gene file: $(basename "$input_file")"

    # Use awk to filter rows with all-zero counts
    awk '
NR == 1 {
    # Always print header
    print $0
    next
}
{
    # Check if any of the count columns (2 to NF) are not zero
    has_counts = 0
    for (i = 2; i <= NF; i++) {
        if ($i != 0) {
            has_counts = 1
            break
        }
    }

    if (has_counts) {
        print $0
    }
}' "$input_file" > "$output_file"
}

# Function to process transcript files (checks cols 3-NF)
process_transcript_file() {
    local input_file="$1"
    local output_file="$2"

    echo "Processing transcript file: $(basename "$input_file")"

    # Use awk to filter rows with all-zero counts
    awk '
NR == 1 {
    # Always print header
    print $0
    next
}
{
    # Check if any of the count columns (3 to NF) are not zero
    has_counts = 0
    for (i = 3; i <= NF; i++) {
        if ($i != 0) {
            has_counts = 1
            break
        }
    }

    if (has_counts) {
        print $0
    }
}' "$input_file" > "$output_file"
}

```

```

    }
    }' "$input_file" > "$output_file"
}

# --- Main Logic ---

echo "Starting zero-count filtering process..."

# Process each input file
for input_file in "${INPUT_FILES[@]"; do
    # Check if input file exists
    if [ ! -f "$input_file" ] || [ ! -r "$input_file" ]; then
        echo "Warning: Input file '$input_file' not found or is not readable. Skipping."
        continue
    fi

    # Generate output filename
    base_name=$(basename "$input_file" .txt)
    output_file="${SCRIPT_DIR}/${base_name}.txt"

    # Determine file type and process accordingly
    if [ "$input_file" == *"counts_gene.txt" ]; then
        process_gene_file "$input_file" "$output_file"
    else
        process_transcript_file "$input_file" "$output_file"
    fi
done

echo "Zero-count filtering process completed."
echo ""
echo "Generated files:"

for input_file in "${INPUT_FILES[@]"; do
    base_name=$(basename "$input_file" .txt)
    output_file="${SCRIPT_DIR}/${base_name}.txt"
    if [ -f "$output_file" ]; then
        echo "  - $(basename "$output_file")"
    fi
done

```

Import R libraries

```

library(tidyverse)
library(ggplot2)
library(scales)
library(RColorBrewer)
library(cowplot)

```

Setup the output files in a dictionary.

```

# Define input files
bambu_outputs <- c(
    gene = "BambuOutput_counts_gene.txt",
    transcript = "BambuOutput_counts_transcript.txt",
    fullLength = "BambuOutput_fullLengthCounts_transcript.txt",
    uniquelyMapped = "BambuOutput_uniqueCounts_transcript.txt"
)

# Initialize storage lists

```

```
results_list <- list()
ids_full <- NULL
ids_unique <- NULL
```

Iterate through each file, count how many features are Annotated vs. Novel, and extract specific IDs for the intersection step.

```
# Loop through each file defined in 'bambu_outputs'
for (name in names(bambu_outputs)) {
  current_file <- bambu_outputs[[name]]

  # Use tryCatch to prevent the entire loop from crashing if one file is missing
  tryCatch({

    # 1. Read the file
    # stringsAsFactors = FALSE is crucial so IDs are read as text, not factors
    data <- read.delim(current_file, header = TRUE, sep = "\t", stringsAsFactors = FALSE)

    # 2. Extract Feature IDs (Column 1)
    # We force them to character to ensure safe text comparison later
    id_column <- as.character(data[[1]])

    # 3. Save IDs for the Intersection Step
    # We specifically need the IDs from 'fullLength' and 'uniquelyMapped'
    # to find the high-confidence overlap in the next cell.
    if (name == "fullLength") ids_full <- id_column
    if (name == "uniquelyMapped") ids_unique <- id_column

    # 4. Calculate Statistics
    # 'startsWith' is a fast base R function to check prefixes
    total_count <- nrow(data)
    annotated_count <- sum(startsWith(id_column, "ENS")) # Ensembl IDs (Known)
    novel_count <- sum(startsWith(id_column, "Bambu")) # Bambu IDs (Novel)

    # 5. Store the results in a temporary dataframe
    results_list[[name]] <- data.frame(
      Bambu = name,
      Total = total_count,
      Annotated = annotated_count,
      Novel = novel_count
    )

  }, error = function(e) {
    # If a file is missing/corrupt, print a warning but continue the loop
    cat("Warning: Could not process '", name, "'. Error: ", e$message, "\n", sep = "")
  })
}
```

Calculate the intersection of transcripts that are **BOTH** full-length and uniquely mapped. This represents the highest confidence subset.

```
# Check if both required ID lists were successfully loaded
if (!is.null(ids_full) && !is.null(ids_unique)) {

  # 1. Find the Intersection
  # 'intersect' returns a vector of values present in both lists
  both_ids <- intersect(ids_full, ids_unique)

  message(paste("Intersection found:", length(both_ids), "high-confidence transcripts."))

  # 2. Calculate Stats for this new group
  results_list[["fullAndUnique"]] <- data.frame(
    Bambu = "fullAndUnique",
```



```

    Total      = length(both_ids),
    Annotated = sum(startsWith(both_ids, "ENS")),
    Novel      = sum(startsWith(both_ids, "Bambu"))
  )
} else {
  warning("Could not calculate intersection. Check if 'fullLength' and 'uniquelyMapped' files exist.")
}

```

Intersection found: 109030 high-confidence transcripts.

Merge the individual results into one master dataframe and reshape it into “Long Format” for `ggplot2`.

```

# 1. Combine all dataframes in the list into one large table
bambu_df <- do.call(rbind, results_list)
rownames(bambu_df) <- NULL # Clean up row names

# 2. Data Transformation Pipeline
bambu_df <- bambu_df %>%
  # Create a 'FeatureType' column to distinguish Genes from Transcripts for faceting
  mutate(FeatureType = if_else(Bambu == "gene", "Gene", "Transcript")) %>%

  # Pivot to Long Format:
  # ggplot prefers one column for X categories (Status) and one for Y values (Count)
  pivot_longer(
    cols      = c(Total, Annotated, Novel),
    names_to  = "Status",
    values_to = "Count"
  )

# 3. Define Factor Levels (Ordering)
# This controls the order of bars in the plot (Total -> Annotated -> Novel)
bambu_df$Status <- factor(bambu_df$Status, levels = c("Total", "Annotated", "Novel"))

# This controls the order of groups on the X-axis
plot_order <- c("gene", "transcript", "fullLength", "uniquelyMapped", "fullAndUnique")
bambu_df$Bambu <- factor(bambu_df$Bambu, levels = plot_order)

# 4. Define Custom Display Labels
# We map the internal code names (e.g., 'fullAndUnique') to pretty labels for the graph
custom_labels <- c(
  "gene"          = "Genes",
  "transcript"    = "All transcripts",
  "fullLength"    = "Full length",
  "uniquelyMapped" = "Uniquely mapped",
  "fullAndUnique" = "Full length &\nUniquely mapped" # \n adds a line break
)

```

Create two separate plots (Gene vs. Transcript) and combine them side-by-side using `cowplot`.

```

# --- PLOT A: GENES ---
bambu_gene_data <- bambu_df %>% filter(FeatureType == "Gene")

bambu_gene_plot <- ggplot(bambu_gene_data, aes(x = Status, y = Count, fill = Status)) +
  geom_col(show.legend = FALSE, position = "dodge") +
  # Add text labels on top of bars
  geom_text(aes(label = Count), vjust = -0.3, size = 4) +
  scale_fill_brewer(palette = "Set1") +
  labs(title = "Genes", x = "\nStatus", y = "No. of genes\n") +
  theme_minimal(base_size = 14) +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))

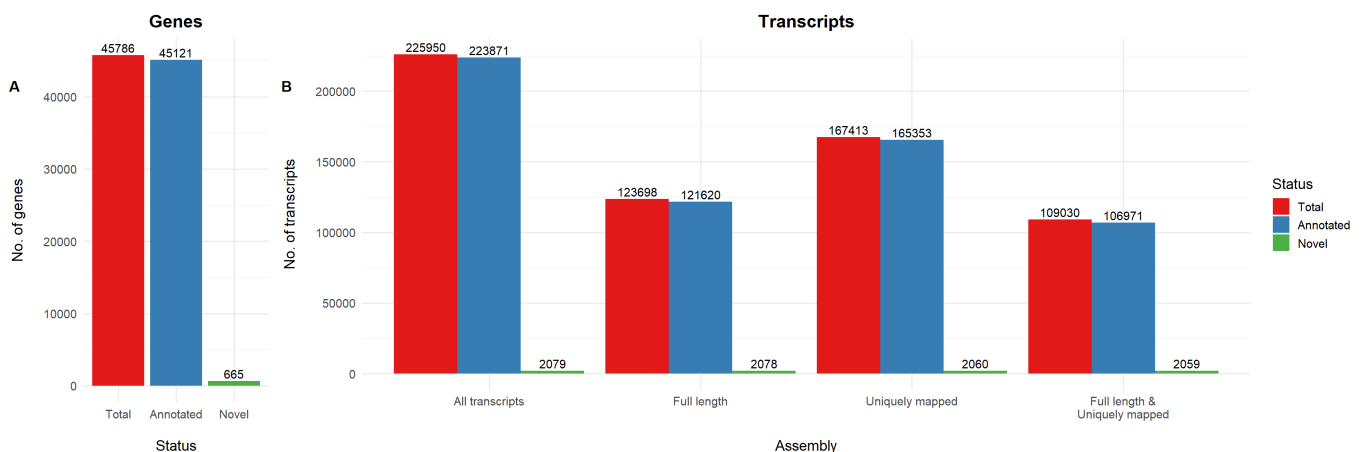
```

```
# --- PLOT B: TRANSCRIPTS ---
bambu_transcript_data <- bambu_df %>%
  filter(FeatureType == "Transcript") %>%
  # Drop unused levels after filtering to avoid 'number of levels differs' error
  mutate(Bambu = droplevels(Bambu))

# Apply the custom pretty labels to the factor levels
# We extract only the levels relevant to transcripts that are now present in bambu_transcript_data$Bambu
levels(bambu_transcript_data$Bambu) <- custom_labels[levels(bambu_transcript_data$Bambu)]

bambu_transcript_plot <- ggplot(bambu_transcript_data, aes(x = Bambu, y = Count, fill = Status)) +
  geom_col(position = "dodge") +
  geom_text(aes(label = Count), position = position_dodge(width = 0.9), vjust = -0.3, size = 4) +
  scale_fill_brewer(palette = "Set1") +
  labs(
    title = "Transcripts",
    x = "\nAssembly",
    y = "No. of transcripts\n",
    fill = "Status"
  ) +
  theme_minimal(base_size = 14) +
  theme(
    plot.title = element_text(hjust = 0.5, face = "bold"),
    axis.text.x = element_text(angle = 0, hjust = 0.5), # Keep x-axis straight
    legend.position = "right"
  )

# --- COMBINE PLOTS ---
# Use plot_grid to arrange them.
# rel_widths makes the Transcript plot wider (1.6) than the Gene plot (0.4)
plot_grid(bambu_gene_plot, bambu_transcript_plot, ncol = 2,
  rel_widths = c(0.4, 1.6),
  labels = "AUTO", label_size = 14, label_y = 0.85)
```



```
# 1. Verify we have the intersection IDs
if (exists("both_ids") && length(both_ids) > 0) {

  message(paste("Filtering count matrix for", length(both_ids), "transcripts..."))

# 2. Load the MAIN transcript count matrix
# This file contains the actual expression counts (CPM/Counts)
df_counts <- read.delim(bambu_outputs["transcript"], header = TRUE, sep = "\t", check.names = FALSE)

# 3. Filter Rows
# We keep only rows where the Transcript ID (Column 1) is in our 'both_ids' list
# df_counts[, 1] assumes the first column is the TXNAME
```

```

df_filtered <- df_counts[df_counts[, 1] %in% both_ids, ]

# 4. Save to Disk
output_filename <- "BambuOutput_counts_transcript_fullAndUnique.tsv"
write.table(df_filtered, output_filename, sep = "\t", quote = FALSE, row.names = FALSE)

message(paste("Success! Filtered matrix saved to:", output_filename))

} else {
  stop("No intersection IDs found. Please run the intersection step (Cell 3) first.")
}

```

Filtering count matrix for 109030 transcripts...

Success! Filtered matrix saved to: BambuOutput_counts_transcript_fullAndUnique.tsv

Now that we have identified the “Full Length & Uniquely Mapped” transcripts, we filter the main count matrix to keep only these high-confidence rows and save it for downstream analysis.

```

# 1. Verify we have the intersection IDs
if (exists("both_ids") && length(both_ids) > 0) {

  message(paste("Filtering count matrix for", length(both_ids), "transcripts..."))

  # 2. Load the MAIN transcript count matrix
  # This file contains the actual expression counts (CPM/Counts)
  df_counts <- read.delim(bambu_outputs["transcript"], header = TRUE, sep = "\t", check.names = FALSE)

  # 3. Filter Rows
  # We keep only rows where the Transcript ID (Column 1) is in our 'both_ids' list
  # df_counts[, 1] assumes the first column is the TXNAME
  df_filtered <- df_counts[df_counts[, 1] %in% both_ids, ]

  # 4. Save to Disk
  output_filename <- "BambuOutput_counts_transcript_fullLength_uniqueCounts.txt"
  write.table(df_filtered, output_filename, sep = "\t", quote = FALSE, row.names = FALSE)

  message(paste("Success! Filtered matrix saved to:", output_filename))

} else {
  stop("No intersection IDs found. Please run the intersection step (Cell 3) first.")
}

```

Filtering count matrix for 109030 transcripts...

Success! Filtered matrix saved to: BambuOutput_counts_transcript_fullLength_uniqueCounts.txt